Aufgabe 1: Kritische Abschnitte

Synchronisation & Kritische Abschnitte – Formelsammlung & Übersicht

Kritischer Abschnitt (Critical Section): Programmabschnitt, in dem auf gemeinsame Ressourcen (Variablen, Dateien, Hardware) zugegriffen wird.
Race Condition (Wettlaufsituation): Fehlerhafter Zustand, bei dem das Endergebnis unkontrolliert von der zeitlichen Ausführungsreihenfolge (Scheduling) der Threads abhängt.
Vier Bedingungen für korrekten wechselseitigen Ausschluss (Tanenbaum):

1. **Mutual Exclusion:** Niemals dürfen sich zwei Prozesse gleichzeitig im kritischen Abschnitt befinden.
2. **No Assumptions:** Keine Annahmen über Ausführungsgeschwindigkeiten oder Prozessoranzahl.
3. **Progress:** Kein Prozess außerhalb des kritischen Abschnitts darf andere blockieren.
4. **Bounded Waiting (Fairness):** Kein Prozess darf unendlich lange auf Einlass warten müssen (kein Verhungern / Starvation).

Aufgabe 1: C#-Threads & Gemeinsame Variable (Shared ID)

Interaktive Simulation von Race Conditions in C#-Threads



Simulator ↗ In

der Vorlesung haben wir folgendes Multithreading-Beispiel behandelt:

```
using System;
using System.Threading;

namespace Threads
{
    class Program
    {
        private const int NumberOfThreads = 10;

        static void StaticMethod(object threadId)
        {
            Console.WriteLine($"Hello World. Greetings from thread {threadId}");
        }

        private static void Main(string[] args)
        {
            for (var i = 0; i < NumberOfThreads; i++)
            {
                Console.WriteLine($"Main here. Creating thread {i}");
                var newThread = new Thread(StaticMethod);
                newThread.Start(i);
            }
        }
    }
}
```

a) Wie müssten Sie das Programm erweitern, sodass ein kritischer Abschnitt entsteht und in der Programmausgabe zumindest ab und zu der Fehler (Race Condition) deutlich wird?

Eine statische gemeinsame Variable (z. B. `static int currentThreadId;`) definieren. In der Thread-Methode diese Variable setzen, eine kurze Pause (`Thread.Sleep(1)`) einlegen und anschließend den Wert der Variablen ausgeben. Durch den Thread-Wechsel während des Schlafs wird die Variable von einem anderen Thread überschrieben, wodurch falsche IDs ausgegeben werden.

b) Wie lässt sich dieser kritische Abschnitt in C# absichern, um die Race Condition vollständig zu verhindern?

Durch ein Sperrobjekt mit `lock`:

```
private static readonly object _lock = new object();
```

Innerhalb der Methode wird der Zugriff geschützt:

```
lock (_lock) { currentThreadId = id; Thread.Sleep(1); Console.WriteLine(currentThreadId);
}
```

Aufgabe 2: Lost Update beim Bankkonto (Read-Modify-Write)

Interaktive Demonstration des Lost-Update-Phänomens **Simulator** ↗

Zwei Threads T_1 und T_2 greifen nebenläufig auf dasselbe Bankkonto mit dem Anfangssaldo **100 €** zu. T_1 hebt **50 €** ab, T_2 hebt **30 €** ab. Die Abhebung besteht aus drei internen Einzelschritten:

1. **Read:** Saldo aus dem Hauptspeicher in das thread-lokale Register R laden (`R = saldo`)
2. **Modify:** Betrag im Register subtrahieren (`R = R - betrag`)
3. **Write:** Neuen Wert aus dem Register in den Hauptspeicher zurückschreiben (`saldo = R`)

Betrachten Sie folgendes unglückliches Interleaving:

Schritt	Aktion	R_1	R_2	Saldo im Speicher
0	Start	–	–	100 €
1	T_1: `R1 = saldo`	100 €	–	100 €
2	T_2: `R2 = saldo`	100 €	100 €	100 €
3	T_1: `R1 = R1 - 50`	50 €	100 €	100 €
4	T_1: `saldo = R1`	50 €	100 €	**50 €**
5	T_2: `R2 = R2 - 30`	50 €	70 €	50 €
6	T_2: `saldo = R2`	50 €	70 €	**70 €**

c) Welcher Endsaldo steht nach Schritt 6 im Speicher, und welcher Betrag hätte bei korrekter Ausführung im Speicher stehen müssen?

Im Speicher steht **70 €**. Korrekt wäre: $100 - 50 - 30 = 20$ €. Die Abhebung von T_1 (50 €) wurde durch das Überschreiben von T_2 vollständig gelöscht (*Lost Update*).

Aufgabe 3: Shared Counter & Assembler-Instruktionen

Interaktives Interleaving auf Maschinenbefehlsebene

Simulator ↗

Eine globale Zählervariable `count` wird von Thread A inkrementiert (`count++`) und von Thread B dekrementiert (`count--`). Auf Assemblerebene besteht der Inkrement-Befehl aus:

- `MOV R1, [count]` (Laden)
- `ADD R1, 1` (Inkrementieren)
- `MOV [count], R1` (Speichern)

Analog dekrementiert Thread B über Register `R2`. Der Anfangswert von `count` ist **5**.

d) Geben Sie alle möglichen Endwerte von `count` nach je einer vollständigen Ausführung beider Operationen an, wenn ein beliebiges Interleaving auftreten kann.

Mögliche Endwerte sind **4, 5, 6**:

- **5 (korrekt):** Wenn die Operationen sequentiell nacheinander ausgeführt werden ($5 + 1 - 1 = 5$).
- **4 (Lost Increment):** A liest 5, B liest 5 und schreibt 4. B gewinnt, Inkrement geht verloren.
- **6 (Lost Decrement):** A liest 5, B liest 5, B schreibt 4, A schreibt 6. Dekrement geht verloren.

Aufgabe 4: Vier Kriterien für wechselseitigen Ausschluss

Tanenbaum-Kriterien für Synchronisationsprimitive



Simulator ↗

Um kritische Abschnitte korrekt abzusichern, müssen Synchronisationsverfahren strenge Kriterien erfüllen.

e) Nennen Sie die vier Bedingungen nach Tanenbaum/Dijkstra für eine korrekte Lösung des Problems des kritischen Abschnitts.

1. **Wechselseitiger Ausschluss (Mutual Exclusion):** Es dürfen sich nie zwei Prozesse gleichzeitig in ihren kritischen Abschnitten befinden.
2. **Keine Annahmen:** Es dürfen keine Annahmen über Ausführungsgeschwindigkeiten oder CPU-Anzahl getroffen werden.
3. **Keine Blockade von außen (Progress):** Kein Prozess außerhalb eines kritischen Abschnitts darf einen anderen Prozess blockieren.
4. **Beschränktes Warten (Bounded Waiting / Fairness):** Kein Prozess darf unendlich lange warten müssen, um in den kritischen Abschnitt zu gelangen (keine Starvation).

f) Was versteht man unter *Busy Waiting* (aktivem Warten) und warum ist es auf Uniprozessorsystemen besonders schädlich?

Beim Busy Waiting verharrt ein Prozess in einer Schleife (z. B. `while(turn != 0);`), bis eine Bedingung wahr wird. Auf einer Uniprozessor-CPU verschwendet dies wertvolle Rechenzeit, da der blockierte Prozess die CPU belegt, während der andere Prozess, der die Bedingung erfüllen könnte, nicht gerechnet werden kann (Prioritätsinversion / CPU-Verschwendung).

Aufgabe 5: Petersons Algorithmus (Zwei-Prozesse-Synchronisation)

Interaktiver Peterson-Simulator im Web **Simulator** ↗

Petersons Algorithmus garantiert wechselseitigen Ausschluss für zwei Prozesse P_0 und P_1 im User-Space ohne Hardware-Unterstützung:

Prozess P_0	**Prozess P_1**
`flag[0] = true;`	`flag[1] = true;`
`turn = 1;`	`turn = 0;`
`while (flag[1] && turn == 1); // warten`	`while (flag[0] && turn == 0); // warten`
`// Kritischer Abschnitt`	`// Kritischer Abschnitt`
`flag[0] = false;`	`flag[1] = false;`

g) Welche Rolle spielen das Array `flag[]` und die Variable `turn`?

`flag[i] = true` signalisiert die **Absicht** von Prozess `i`, den kritischen Abschnitt zu betreten. `turn` regelt das **Vorfahrtsrecht** bei gleichzeitigem Eintrittswunsch (Höflichkeitsprinzip: Jeder Prozess überlässt dem anderen zunächst den Vortritt).

h) Warum funktioniert Petersons Algorithmus auf modernen Mehrkern-CPUs nicht mehr ohne Weiteres ohne *Memory Barriers* (Speicherbarrieren)?

Moderne CPUs und Compiler führen Befehle zur Leistungssteigerung *Out-of-Order* (in veränderter Reihenfolge) aus und puffern Schreiboperationen im Store Buffer. Dadurch kann ein Kern das Setzen von `flag` verzögert sehen, sodass beide Prozesse gleichzeitig die Eintrittsbedingung als erfüllt ansehen.